PES-VCS: A Version Control System

Operating Systems Lab — Unit 4

R HEMANTH REDDY
SRN: PES1UG24AM390

GitHub: https://github.com/hemanthloq/PES1UG24AM390-pes-vcs

Phase 1 — Object Store

The object store is the core of PES-VCS. Every file, directory snapshot, and commit is stored as a content-addressable object named by its SHA-256 hash. Objects are stored under .pes/objects/XX/YYY... where XX is the first two hex characters (directory sharding). The two key functions implemented are object_write (stores data with a type header and hash-based path) and object_read (retrieves and verifies integrity via hash comparison).

Screenshot 1A — Phase 1 tests passing

```
osboxes@HEMANTHREDDYPES1UG24AM390:~/Desktop/PES1UG24AM390-p
es-vcs$ ./pes init
./test_objects
Initialized empty PES repository in .pes/
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09
227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7
d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
osboxes@HEMANTHREDDYPES1UG24AM390:~/Desktop/PES1UG24AM390-p
es-vcs$
```

Screenshot 1A: ./test_objects — All Phase 1 tests passed

Screenshot 1B — Object store directory structure

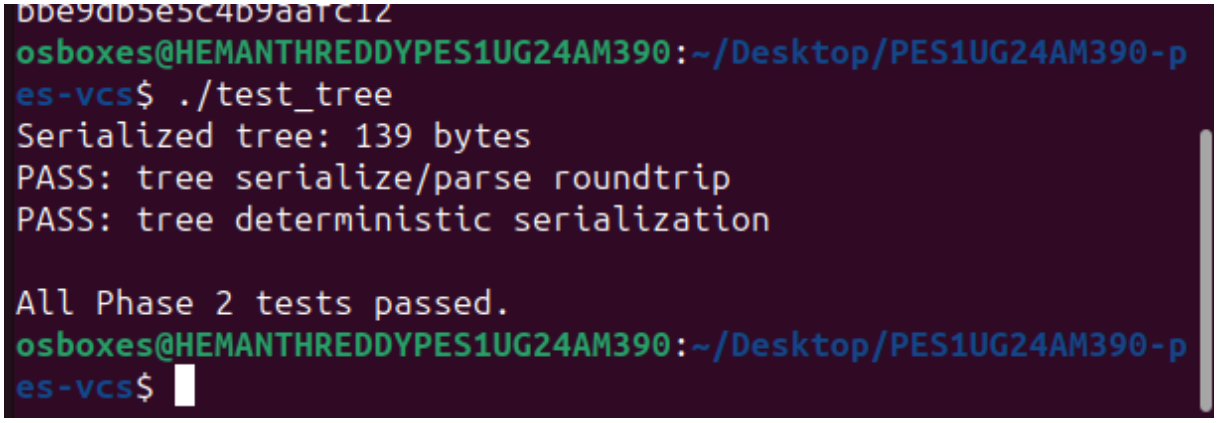
```
osboxes@HEMANTHREDDYPES1UG24AM390:~/Desktop/PES1UG24AM390-p
es-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aa
f2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb
9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221
bbe9db5e5c4b9aafc12
osboxes@HEMANTHREDDYPES1UG24AM390:~/Desktop/PES1UG24AM390-p
es-vcs$
```

Screenshot 1B: find .pes/objects -type f — sharded directory layout

Phase 2 — Tree Objects

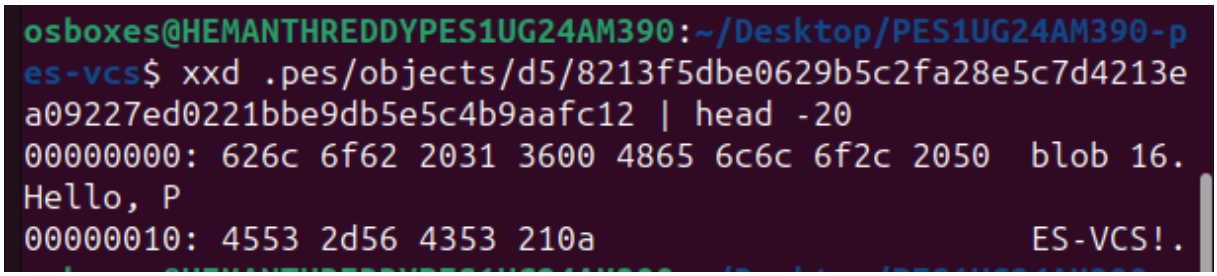
Tree objects represent directory snapshots. Each tree entry stores a file mode (octal), filename, and 32-byte binary SHA-256 hash. Entries are sorted alphabetically before serialization to ensure deterministic hashing — identical directory contents always produce the same tree hash. tree_serialize converts a Tree struct to binary format; tree_parse reads it back.

Screenshot 2A — Phase 2 tests passing



Screenshot 2A: ./test_tree — All Phase 2 tests passed

Screenshot 2B — Raw binary xxd dump of a tree object



Screenshot 2B: xxd of a raw object file showing blob header and content

Phase 3 — Index / Staging Area

The index is a text file (.pes/index) that tracks staged files before a commit. Each line stores: mode, SHA-256 hex hash, mtime, size, and path. index_load parses this file into memory; index_save writes it atomically using a temp file and rename to prevent corruption; index_add reads a file, writes its blob to the object store, and updates the index entry.

Screenshot 3A — pes init, pes add, pes status

```
osboxes@HEMANTHREDDYPES1UG24AM390:~/Desktop/PES1UG24AM390-p
es-vcs$ ./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  tree.h
  untracked:  object.c
  untracked:  test_tree.c
  untracked:  Makefile
  untracked:  README.md
  untracked:  index.h
  untracked:  commit.h
  untracked:  pes.c
  untracked:  index.c
  untracked:  test_objects.c
  untracked:  pes.h
  untracked:  commit.c
  untracked:  test_tree
  untracked:  test_objects
  untracked:  tree.c
  untracked:  test_sequence.sh
  untracked:  .gitignore
```

Screenshot 3A: ./pes init → ./pes add file1.txt file2.txt → ./pes status

Screenshot 3B — Contents of .pes/index

```
osboxes@HEMANTHREDDYPES1UG24AM390:~/Desktop/PES1UG24AM390-p
es-vcs$ cat .pes/index
100664 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025
de576db9ebb4 1776335725 6 file1.txt
100664 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65
d6f606199f7f 1776335725 6 file2.txt
osboxes@HEMANTHREDDYPES1UG24AM390:~/Desktop/PES1UG24AM390-p
es-vcs$
```

Screenshot 3B: cat .pes/index — text-format index showing staged entries

Phase 4 — Commits and History

commit_create builds a commit object by first writing a tree from the index, reading the current HEAD as parent (empty for first commit), recording the author and timestamp, serializing the commit struct to text format, writing it to the object store, and finally updating HEAD to point to the new commit hash. commit_walk traverses the parent chain to implement pes log.

Screenshot 4A — pes log showing three commits

```
osboxes@HEMANTHREDDYPES1UG24AM390:~/Desktop/PES1UG24AM390-p
es-vcs$ ./pes commit -m "Initial commit"
echo "Hello again" >> file1.txt
./pes add file1.txt
./pes commit -m "Update file1"
echo "bye" > file3.txt
./pes add file3.txt
./pes commit -m "Add file3"
./pes log
Committed: 465f46deea29... Initial commit
Committed: 14384529005b... Update file1
Committed: 7f8f4c341dcd... Add file3
commit 7f8f4c341dcd84e1212e071a0e4dfdb8aed5a2ec07e4e3809a68
aa68598fe78c
Author: R HEMANTH REDDY <PES1UG24AM390>
Date:   1776335879

    Add file3

commit 14384529005b516cc9e33842362d3fe963d63564af5fad0bf3d3
59f8313f8c9d
Author: R HEMANTH REDDY <PES1UG24AM390>
Date:   1776335879

    Update file1r auth!

commit 465f46deea2916646d12b8de97a3c0cf3601b75257fe7176fa66
841af05d5ed1
Author: R HEMANTH REDDY <PES1UG24AM390>
Date:   1776335879

    Initial committial commitEA
```

Screenshot 4A: ./pes log — three commits with hashes, authors, timestamps

Screenshot 4B — Object store growth after commits

```
osboxes@HEMANTHREDDYPES1UG24AM390:~/Desktop/PES1UG24AM390-p
es-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/14/384529005b516cc9e33842362d3fe963d63564af5fa
d0bf3d359f8313f8c9d
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d33
7639025de576db9ebb4
.pes/objects/46/5f46deea2916646d12b8de97a3c0cf3601b75257fe7
176fa66841af05d5ed1
.pes/objects/5c/4e1e2f5a561dd416ca159203c3259111f64259b7341
e2754484f9b8209b0ae
.pes/objects/7f/8f4c341dcd84e1212e071a0e4dfdb8aed5a2ec07e4e
3809a68aa68598fe78c
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35
f9f7e65d6f606199f7f
.pes/objects/e9/ec68a838da3a14f2ea8d7f5fb54c3921e54c0ed4a0f
765e189019432f74c52
.pes/objects/eb/d32ede2dc675fbfaeaa25d98d07732e14f2053e04c3
fe72588c894b10d7f1a
.pes/refs/heads/main
osboxes@HEMANTHREDDYPES1UG24AM390:~/Desktop/PES1UG24AM390-p
es-vcs$
```

Screenshot 4B: find .pes -type f | sort — all objects and refs

Screenshot 4C — HEAD and refs/heads/main

```
osboxes@HEMANTHREDDYPES1UG24AM390:~/Desktop/PES1UG24AM390-p
es-vcs$ cat .pes/refs/heads/main
cat .pes/HEAD
7f8f4c341dcd84e1212e071a0e4dfdb8aed5a2ec07e4e3809a68aa68598
fe78c
ref: refs/heads/main
```

Screenshot 4C: cat .pes/refs/heads/main and cat .pes/HEAD

Final — Integration Test

The full integration test (make test-integration) runs test_sequence.sh which performs end-to-end testing: repository initialization, staging files, making commits, and verifying commit history. All phases are tested together.

Final Screenshot Part 1 — Integration test output

```
To https://github.com/hemanthloq/PES1UG24AM390-pes-vcs.git
   be42926..a4fd379  main -> main
osboxes@HEMANTHREDDYPES1UG24AM390:~/Desktop/PES1UG24AM390-pes-vcs$ cd ~/Desktop/PES1UG24AM390-pes-vcs
make all
export PES_AUTHOR="R HEMANTH REDDY <PES1UG24AM390>"
make test-integration
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
index.c: In function ‘index_add’:
index.c:166:9: warning: implicit declaration of function ‘object_write’ [-Wimplicit-function-declaration]
  166 |     if (object_write(OBJ_BLOB, data, size, &id) != 0) {
      |         ^~~~~~~~~~~
index.c:162:5: warning: ignoring return value of ‘fread’ declared with attribute ‘warn_unused_result’ [-Wunused-result]
  162 |     fread(data, 1, size, f);
      |     ^~~~~~~~~~~~~~~~~~~~~~~
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt
```

Final Screenshot (Part 1): make test-integration — repository init and staging

Final Screenshot Part 2 — Integration test continued

```
  (nothing to snow)


--- First Commit ---
Committed: 286a75d9bcde... Initial commit

Log after first commit:
commit 286a75d9bcdef20e869561a63c4620af3501234ef39a1c5e7a5be4ec94e26e0c
Author: R HEMANTH REDDY <PES1UG24AM390>
Date:   1776340542

    Initial commit


--- Second Commit ---
Committed: b1089784ad64... Update file.txt

--- Third Commit ---
Committed: c729c5c5699a... Add farewell

--- Full History ---
commit c729c5c5699a07513191e443b80efdbae17e6ef2b2537e9df385bdf106bec304
Author: R HEMANTH REDDY <PES1UG24AM390>
Date:   1776340542

    Add farewellr auth!

commit b1089784ad64f8dbd9d24510cd5b9d5c1e3aec5d4631efd0584e07645d521c51
Author: R HEMANTH REDDY <PES1UG24AM390>
Date:   1776340542

    Update file.txtuth!

commit 286a75d9bcdef20e869561a63c4620af3501234ef39a1c5e7a5be4ec94e26e0c
```

Final Screenshot (Part 2): make test-integration — commit history verification

Phase 5 & 6 — Written Analysis

Q5.1 — How would you implement pes checkout ?

To implement checkout, three things must change in .pes/: (1) HEAD must be updated to write "ref: refs/heads/" into .pes/HEAD. (2) The index must be updated to reflect the target branch's tree. (3) Every tracked file in the working directory must be replaced with the version from the target branch's tree, by reading the commit pointed to by the branch, reading the root tree object, and recursively writing each blob back to the filesystem. This is complex because: files modified since last staging must be detected and checkout must refuse if they differ between branches; subdirectories must be created or deleted as needed; and if checkout fails halfway the repository ends up in an inconsistent state, so changes must be applied atomically.

Q5.2 — How would you detect a dirty working directory conflict?

For each entry in the index, use stat() to compare mtime and size against the stored metadata. If either differs, re-hash the actual file on disk and compare its blob hash to the hash stored in the index. If they differ, the file is genuinely dirty (not just a timestamp change). Then read the target branch's root tree from the object store and check whether that dirty file also differs from what the target branch expects. If a dirty file differs from both the current index AND the target tree, checkout must refuse. If the dirty content already matches the target branch, checkout can safely proceed for that file. This algorithm uses only the index for metadata and the object store for blob hash comparison — no additional data structures needed.

Q5.3 — What happens in detached HEAD state and how to recover?

In detached HEAD state, HEAD contains a raw commit hash instead of a branch reference like "ref: refs/heads/main". New commits are created normally — each has the previous commit as parent and HEAD is updated to point directly to the new commit hash. However, no branch reference file is updated, so these commits are not reachable from any named branch. If you switch to a different branch, those commits appear lost. To recover: if still in detached state, create a branch immediately: "git branch recovery-branch" which saves the current HEAD hash as a branch. If already switched away, find the orphaned commit hash from terminal scroll-back history or reflog, then run "git branch recovered " to create a branch pointing to those commits before garbage collection removes them.

Q6.1 — Algorithm to find and delete unreachable objects

Use mark-and-sweep garbage collection: (1) MARK PHASE — collect all root references by reading every file under .pes/refs/heads/ to get all branch tip commit hashes, plus HEAD. Perform a BFS/DFS traversal from each root commit: read the commit object and mark it reachable, extract its tree hash and follow it, read the tree object and mark it reachable, recursively mark all blob and subtree hashes reachable, then follow the parent pointer to the previous commit and repeat. (2) SWEEP PHASE — walk all files under .pes/objects/ using opendir/readdir. For each object file, reconstruct its hash from the directory and filename. If the hash is NOT in the reachable set, delete the file. DATA STRUCTURE: A hash set keyed on the 32-byte ObjectID gives O(1) average membership check. ESTIMATE: For 100,000 commits and 50 branches, assuming 1 commit + 10 trees + 20 blobs per commit = 31 objects per commit, total objects to visit ≈ 3,100,000. Many objects are shared across commits so actual unique visits are lower due to deduplication in the reachable set.

Q6.2 — Race condition between GC and a concurrent commit

Consider this interleaving: (1) GC begins its mark phase, traversing all reachable objects from current branch tips. (2) A concurrent commit operation starts: it writes a new blob object (blob_new) to the object store, then begins building a tree referencing blob_new. (3) GC finishes its mark phase. blob_new was written after GC's traversal started, so GC never visited it and considers it unreachable. (4) GC sweeps and deletes blob_new. (5) The commit operation finishes writing its tree (which references blob_new) and creates the commit object. The repository now contains a commit that references a deleted blob — permanent data corruption. HOW GIT AVOIDS THIS: Git uses a grace period — objects written within the last 2 weeks (configurable via gc.pruneExpire) are never deleted by GC even if unreachable. Since any commit operation completes in seconds, all newly written objects are protected. Git also uses lock files (.git/index.lock, .git/gc.pid) to prevent concurrent GC runs and to coordinate with other git processes accessing the repository simultaneously.